Running a Local LLM: A Short Practical Guide

Fictional sample document used to demonstrate the document Q&A system.

1. Why run a model locally?

A local language model runs on hardware that the organisation controls. Prompts and documents never leave the machine, which removes the risk of confidential data reaching a third-party service. Local models also work without an internet connection and have no per-request fees. The trade-off is that smaller local models are less capable than the largest hosted models, and the organisation is responsible for hardware and updates.

2. Model size and memory

The memory needed depends mainly on the number of parameters and on the precision of the weights. A model with 7 billion parameters stored in 4-bit quantised form needs roughly 4 to 5 GB of memory, while the same model at 16-bit precision needs about 14 GB. A 3 billion parameter model in 4-bit form runs comfortably on a laptop with 8 GB of RAM. Quantisation reduces memory use and speeds up inference at a small cost in answer quality.

3. Context window

The context window is the maximum amount of text, measured in tokens, that a model can read in one request. Both the retrieved document passages and the question must fit inside it. Many local runtimes default to a small window of 2,048 or 4,096 tokens, so the window should be increased explicitly when long passages are inserted. A larger context window needs more memory, because the attention cache grows with the number of tokens.

4. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation combines a search step with a language model. Documents are split into chunks, each chunk is converted into an embedding vector, and the vectors are stored in a vector database. When a user asks a question, the question is embedded too, the most similar chunks are retrieved, and they are given to the model as context. Because the answer is grounded in the retrieved text, RAG greatly reduces hallucinations and allows the model to cite its sources. It also avoids retraining: new documents are simply added to the index.

5. Tuning a RAG system

Three settings have the largest effect on quality. The chunk size controls how much text each embedding represents; chunks of about 500 to 1,000 characters are a good starting point. The chunk overlap, often 10 to 20 percent of the chunk size, prevents answers from being cut at chunk boundaries. The top-k value sets how many chunks are given to the model: too few can miss the answer, too many dilute the context and add noise. These values should be tested on a set of example questions rather than guessed.

6. Evaluating retrieval

Retrieval quality can be measured before involving the language model. For each test question, the expected source passage is known. Hit rate at k is the share of questions for which the correct passage appears among the top k results. Mean reciprocal rank (MRR) rewards ranking the correct passage higher. Answer quality is checked separately by comparing generated answers with reference answers and by confirming that citations point to the right passages.

7. Common failure modes

Typical problems include chunks that are too large, documents with no text layer such as scanned images, tables that lose their structure when converted to text, and questions whose answers are spread across many documents. Scanned PDFs need optical character recognition (OCR) before they can be indexed. A similarity threshold helps the system say that it cannot find an answer instead of guessing.

8. Serving options

Ollama is a simple way to download and run open models on a single computer and exposes a local HTTP API. For higher throughput on a server with a GPU, vLLM provides an OpenAI-compatible API with continuous batching. Both options keep data on infrastructure controlled by the organisation when they are deployed on-premise.